



BLM4522 Ağ Tabanlı Paralel Dağıtım Sistemleri

**Veritabanı Performans Optimizasyonu ve İzleme,
Veri Temizleme ve ETL Süreçlerinin Tasarımı,
Veritabanı Yedekleme ve Otomasyon Çalışması**

GitHub: <https://github.com/ismailbsrn/DatabaseScenarios-3>

GitHub: <https://github.com/ismailbsrn/DatabaseScenarios-4>

GitHub: <https://github.com/ismailbsrn/DatabaseScenarios-5>

Video:

İsmail Başaran

22290137

Mayıs 2026

1. Veritabanı Performans Optimizasyonu ve İzleme

Raporun birinci bölümünde veritabanı performans optimizasyonu ve izleme senaryosu ele alınmaktadır.

1.1. Projenin Amacı

Bu projenin temel amacı, yapay şekilde büyük bir veri tabanı oluşturmak ve bu veri tabanı üzerinde gerçekleştirilen sorguların performansının izlenmesi ve optimize edilmesidir. Veri tabanı yönetim sistemi olarak PostgreSQL kullanılacaktır.

1.2. Proje Kapsamı

Proje aşağıdaki özellikleri içermektedir:

- PostgreSQL için pg_stat_statements eklentisi ile performans takibi.
- Sorgu performansı için ihtiyaç duyulan yerlerde indeks kullanımı.
- Uzun süren sorguları inceleme ve optimize etme.
- Farklı roller için erişim yönetimi.

1.3. Gerçekleştirim ve Yöntem

Bahsedilen aşamaların nasıl gerçekleştirildiği detaylarıyla birlikte bu bölümde açıklanmaktadır.

1.3.1. Veritabanı İzleme

Bu adımda yapay veriler ile büyük bir veri tabanı hazırlanmış ve bu veri tabanı üzerinde gerçekleştirilen sorguların performansı izlenmiştir.

```
1 CREATE TABLE transactions (  
2   id SERIAL PRIMARY KEY,  
3   customer_id INT,  
4   transaction_date TIMESTAMP,  
5   amount NUMERIC(10, 2),  
6   status VARCHAR(20)  
7 );  
8 INSERT INTO transactions (customer_id, transaction_date, amount, status)  
9 SELECT  
10  floor(random() * 100000 + 1)::int,  
11  timestamp '2023-01-01 00:00:00' + random() * (timestamp '2026-01-01 00:00:00' - timestamp '2023-01-01 00:00:00'),  
12  (random() * 10000)::numeric(10, 2),  
13  (ARRAY['success', 'failed', 'pending', 'refunded'])[floor(random() * 4 + 1)]  
14 FROM generate_series(1, 10000000);
```

Yukarıdaki sql scripti ile 10 Milyon satırlık bir tablo oluşturulmuştur.

```
DatabaseScenarios-3 $ psql -U ismailbasaran -d ornek_veritabani < create_db.sql
CREATE TABLE
INSERT 0 10000000
DatabaseScenarios-3 $ psql -U ismailbasaran -d ornek_veritabani
psql (18.3 (Homebrew))
Type "help" for help.

ornek_veritabani=# SELECT pg_size_pretty(pg_relation_size('transactions')) AS disk_boyutu, count(*) AS toplam_satir
FROM transactions;
 disk_boyutu | toplam_satir 
-----+-----
 574 MB      | 100000000
(1 row)

ornek_veritabani=#
```

Yukarıdaki görüntüde görüleceği üzere transactions tablosu 10 Milyon satırdan oluşmaktadır ve diskte 574 Mb yer kaplamaktadır. Performans izleme işlemi için pg_stat_statements eklentisinin kurulması gerekmektedir.

```
ornek_veritabani=# CREATE EXTENSION IF NOT EXISTS pg_stat_statements;
CREATE EXTENSION
```

Yukarıdaki komut ile pg_stat_statements eklentisi kurulmuştur. Bu noktadan sonra veri tabanı üzerinde çalıştırılan sorguların performansı izleme altındadır.

```
ornek_veritabani=# EXPLAIN ANALYZE
SELECT count(*)
FROM transactions
WHERE status = 'failed' AND amount > 9499;

QUERY PLAN
-----
Finalize Aggregate  (cost=137159.31..137159.32 rows=1 width=8) (actual time=209.540..210.523 rows=1.00 loops=1)
  Buffers: shared hit=12111 read=61419
    -> Gather  (cost=137159.10..137159.31 rows=2 width=8) (actual time=209.488..210.518 rows=3.00 loops=1)
          Workers Planned: 2
          Workers Launched: 2
          Buffers: shared hit=12111 read=61419
            -> Partial Aggregate  (cost=136159.10..136159.11 rows=1 width=8) (actual time=203.786..203.786 rows=1.00 loops=3)
                  Buffers: shared hit=12111 read=61419
                    -> Parallel Seq Scan on transactions  (cost=0.00..136030.50 rows=51438 width=0) (actual time=0.213..202.312 rows=41814.33 loops=3)
                          Filter: ((amount > '9499'::numeric) AND ((status)::text = 'failed'::text))
                          Rows Removed by Filter: 3291519
                          Buffers: shared hit=12111 read=61419
Planning Time: 0.154 ms
Execution Time: 210.607 ms
(14 rows)
```

Sonraki aşamalarda görmek üzere uzun zaman alacak yukarıdaki rapor sorgusu çalıştırılmıştır.

```
ornek_veritabani=# SELECT query, calls, round(total_exec_time::numeric, 2) AS total_time_ms, rows
FROM pg_stat_statements
ORDER BY total_exec_time DESC LIMIT 5;
 query | calls | total_time_ms | rows 
-----+-----+-----+-----
 EXPLAIN ANALYZE | 1 | 210.81 | 0
 SELECT count(*) |  |  | 
 FROM transactions |  |  | 
 WHERE status = $1 AND amount > $2 |  |  | 
 SELECT pg_stat_statements_reset() | 1 | 0.50 | 1
(2 rows)
```

Ardından yukarıdaki sorgu ile veri tabanı üzerinde gerçekleştirilen ve en çok zaman alan 5 sorgu ekrana yazdırılmıştır. Görüleceği çalıştırılan rapor sorgusu 210ms sürmektedir. Aktif olmayan örnek bir veri tabanı üzerinde çalışıldığından detaylı bir analiz yapılamıyor olsa da pg_stat_statements aracı ile bu izleme işleminin gerçekleştirilebileceği gösterilmiştir.

1.3.2. Index Yönetimi

Raporun bu bölümünde bir önceki aşamada gerçekleştirilen rapor sorgusunu hızlandırmak için tablodaki status ve amount sütunlarına Composite Index eklenecektir.

```
ornek_veritabani=# CREATE INDEX idx_transactions_status_amount ON transactions(status, amount);
CREATE INDEX
ornek_veritabani=#
```

Bu noktada performansı optimize etmek için yukarıdaki sorgu ile status ve amount sütunlarına Composite Indexleme işlemi yapılmıştır.

```
ornek_veritabani=# SELECT pg_stat_statements_reset();
pg_stat_statements_reset
-----
2026-05-06 11:23:42.760895+03
(1 row)

ornek_veritabani=# EXPLAIN ANALYZE
SELECT count(*)
FROM transactions
WHERE status = 'failed' AND amount > 9499;

                                QUERY PLAN
-----
Aggregate  (cost=4682.16..4682.17 rows=1 width=8) (actual time=35.478..35.479 rows=1.00 loops=1)
  Buffers: shared hit=67717 read=1
  -> Index Only Scan using idx_transactions_status_amount on transactions  (cost=0.56..4373.54 rows=123449 width=0) (actual time=0.153..26.419 rows=125443.00 loops=1)
    Index Cond: ((status = 'failed'::text) AND (amount > '9499'::numeric))
    Heap Fetches: 0
    Index Searches: 1
    Buffers: shared hit=67717 read=1
Planning Time: 0.177 ms
Execution Time: 35.515 ms
(9 rows)
```

Indexleme işlemi sonrasında aynı sorgu tekrar çalıştırılacağından ne kadar sürdüklerini ayrı ayrı görebilmek için pg_stat_statements eklentisinin hafızası resetlenmiş ve sonrasında aynı sorgu tekrar gerçekleştirilmiştir.

```
ornek_veritabani=# SELECT query, calls, round(total_exec_time::numeric, 2) AS total_time_ms, rows
FROM pg_stat_statements
ORDER BY total_exec_time DESC LIMIT 5;

      query      | calls | total_time_ms | rows
-----+-----+-----+-----
EXPLAIN ANALYZE  | 1     | 35.78         | 0
SELECT count(*)  | 1     | 0.58          | 1
FROM transactions | 1     | 0.58          | 1
WHERE status = $1 AND amount > $2 | 1     | 0.58          | 1
SELECT pg_stat_statements_reset() | 1     | 0.58          | 1
(2 rows)
```

Bu sorgular sonrasında yukarıdaki performans izleme sorgusu çağırılmıştır. Görselde görüleceği üzere az önce index yokken 210ms süren sorgu artık 35ms'de gerçekleştirilmektedir. Böylelikle veri tabanının optimize edildiği görülmektedir.

1.3.3. Sorgu İyileştirme

Raporun bu aşamasında sorgu iyileştirmeye yönelik senaryomuz şu şekildedir:

- 2024 yılında yapılan tüm başarılı transactionların getirilmesi istenmektedir.
- transaction_date sütununda bir index mevcuttur.

Tablonun mevcut hali aşağıdaki gibidir.

```
ornek_veritabani=# \d transactions
```

Column	Type	Table "public.transactions"	Collation	Nullable	Default
id	integer			not null	nextval('transactions_id_seq'::regclass)
customer_id	integer				
transaction_date	timestamp without time zone				
amount	numeric(10,2)				
status	character varying(20)				

Indexes:

- "transactions_pkey" PRIMARY KEY, btree (id)
- "idx_transactions_date" btree (transaction_date)
- "idx_transactions_status_amount" btree (status, amount)

Görüleceği üzere işlem tarihleri transaction_date sütununda timestamp olarak saklanmaktadır. Bundan dolayı belirli bir yıldaki işlemlere erişmek için en doğru yol PostgreSQL'in EXTRACT fonksiyonunu kullanmak gibi durmaktadır.

```
ornek_veritabani=# EXPLAIN ANALYZE
SELECT sum(amount)
FROM transactions
WHERE status = 'success'
AND EXTRACT(YEAR FROM transaction_date) = 2024;
```

QUERY PLAN

```
-----
Finalize Aggregate  (cost=147460.08..147460.09 rows=1 width=32) (actual time=248.874..249.866 rows=1.00 loops=1)
  Buffers: shared hit=12797 read=60733
  -> Gather  (cost=147459.86..147460.07 rows=2 width=32) (actual time=248.817..249.858 rows=3.00 loops=1)
    Workers Planned: 2
    Workers Launched: 2
    Buffers: shared hit=12797 read=60733
    -> Partial Aggregate  (cost=146459.86..146459.87 rows=1 width=32) (actual time=242.317..242.318 rows=1.00 loops=3)
      Buffers: shared hit=12797 read=60733
      -> Parallel Seq Scan on transactions  (cost=0.00..146446.67 rows=5276 width=6) (actual time=0.259..229.874 rows=278181.67 loops=3)
        Filter: (((status)::text = 'success'::text) AND (EXTRACT(year FROM transaction_date) = '2024'::numeric))
        Rows Removed by Filter: 3055152
        Buffers: shared hit=12797 read=60733

Planning:
  Buffers: shared hit=31 read=3
Planning Time: 2.353 ms
Execution Time: 249.918 ms
(16 rows)
```

2024 yılında gerçekleştirilen transactionlara erişmek için yukarıdaki sorgu çalıştırıldığında transaction_date sütunu üzerine fonksiyon çağırılmaktadır.

Bundan dolayı sütun üzerinde index olsa dahi indexten faydalanmak mümkün değildir.

```
Finalize Aggregate (cost=147460.08..147460.09 rows=1 width=32) (actual time=248.874..249.866 rows=1.00 loops=1)
  Buffers: shared hit=12797 read=60733
  -> Gather (cost=147459.86..147460.07 rows=2 width=32) (actual time=248.817..249.858 rows=3.00 loops=1)
    Workers Planned: 2
    Workers Launched: 2
    Buffers: shared hit=12797 read=60733
    -> Partial Aggregate (cost=146459.86..146459.87 rows=1 width=32) (actual time=242.317..242.318 rows=1.00 loops=3)
      Buffers: shared hit=12797 read=60733
      -> Parallel Seq Scan on transactions (cost=0.00..146446.67 rows=5276 width=6) (actual time=0.259..229.874 rows=278181.67 loops=3)
        Filter: (((status)::text = 'success'::text) AND (EXTRACT(year FROM transaction_date) = '2024'::numeric))
        Rows Removed by Filter: 3055152
        Buffers: shared hit=12797 read=60733
Planning:
  Buffers: shared hit=31 read=3
Planning Time: 2.353 ms
Execution Time: 249.918 ms
```

Yukarıdaki görselde görüleceği üzere tablo üzerinde Seq Scan işlemi gerçekleştirilmiş ve indexten faydalanılamadığı için sorgu 250ms sürmüştür. Indexten faydalanabilmek için sorgumuzda index bulunan sütun üzerinde fonksiyon kullanmamak gerekmektedir. Bu doğrultuda sorgumuz aşağıdaki gibi olmalıdır.

```
SELECT sum(amount)
FROM transactions
WHERE status = 'success'
AND transaction_date >= '2024-01-01 00:00:00'
AND transaction_date < '2025-01-01 00:00:00';
```

Bu sorgu analiz edilecek olursa aşağıdaki çıktı elde edilecektir.

```
ornek_veritabani=# EXPLAIN ANALYZE
SELECT sum(amount)
FROM transactions
WHERE status = 'success'
AND transaction_date >= '2024-01-01 00:00:00'
AND transaction_date < '2025-01-01 00:00:00';

QUERY PLAN

-----
Finalize Aggregate (cost=33780.16..33780.17 rows=1 width=32) (actual time=83.521..84.481 rows=1.00 loops=1)
  Buffers: shared hit=495137 read=4115
  -> Gather (cost=33779.94..33780.15 rows=2 width=32) (actual time=83.475..84.475 rows=3.00 loops=1)
    Workers Planned: 2
    Workers Launched: 2
    Buffers: shared hit=495137 read=4115
    -> Partial Aggregate (cost=32779.94..32779.95 rows=1 width=32) (actual time=74.081..74.081 rows=1.00 loops=3)
      Buffers: shared hit=495137 read=4115
      -> Parallel Index Only Scan using idx_transactions_cover on transactions (cost=0.56..31895.98 rows=353585 width=6) (actual time=0.460..57.843 rows=278181.67 loops=3)
        Index Cond: ((status = 'success'::text) AND (transaction_date >= '2024-01-01 00:00:00'::timestamp without time zone) AND (transaction_date < '2025-01-01 00:00:00'::timestamp without time zone))
        Heap Fetches: 0
        Index Searches: 1
        Buffers: shared hit=495137 read=4115
Planning:
  Buffers: shared hit=24 read=1
Planning Time: 0.892 ms
Execution Time: 84.521 ms
(17 rows)
```

Yukarıdaki görselde sorgu artık 250ms değil 84ms sürmektedir.

1.3.4. Veri Yöneticisi Roller

Projenin bu adımı vize için yapılan 3. proje kapsamında gerçekleştirilmiştir.

1.4. Çıktılar

Proje kapsamında yapılan işlemlerde görüleceği üzere veri tabanı performans optimizasyonlarında sadece indeks kullanımı yeterli değildir. Veri tabanları üzerinde gerçekleştirilen sorgular uygun araçlar yardımıyla analiz edilmeli, doğru yerde doğru indeksler kullanılmalı ve sorgular bu indekslere uygun olarak hazırlanmalıdır.

2. Veri Temizleme ve ETL Süreçlerinin Tasarımı

Raporun ikinci bölümünde veri temizleme ve süreçleri tasarımı ele alınmaktadır.

2.1. Projenin Amacı

Projenin amacı büyük veri kümelerinin temizlenmesi ve işlenmesi için ETL süreçlerinin tasarlanmasıdır.

2.2. Projenin Kapsamı

Projenin kapsamı şu şekildedir:

- SQL kullanarak hatalı verilerin (örneğin, eksik, tutarsız, ya da yanlış formatta verilerin) temizlenmesi.
- Farklı kaynaklardan gelen verilerin standartlaştırılması ve dönüştürülmesi.
- Verilerin doğru hedef veri tabanlarına yüklenmesi
- Veri temizleme ve dönüştürme sürecine dair raporların oluşturulması.

2.3. Gerçekleştirim ve Yöntem

Bahsedilen aşamaların nasıl gerçekleştirildiği detaylarıyla birlikte bu bölümde açıklanmaktadır.

2.3.1. Veri Temizleme ve Dönüştürme

Veri temizleme ve dönüştürme senaryosu kapsamında `ornek_veritabani` adında bir veri tabanı oluşturulmuştur. Ardından aşağıdaki SQL scripti aracılığıyla kaynağı simüle edecek bir `source_data` şeması ve hedef veri tabanını simüle edecek bir `dest_db` şeması oluşturulmuştur. Sonrasında kirli kaynak

senaryosuna uygun şekilde veri tipleri uygun olmayan, içerisinde boşluklar ve hatalı veriler barındıran içerikler source_data şemasına yazılmıştır.

```
1 CREATE SCHEMA IF NOT EXISTS source_data;
1 CREATE SCHEMA IF NOT EXISTS dest_db;
2
3 CREATE TABLE source_data.customers (
4     id SERIAL PRIMARY KEY,
5     full_name VARCHAR(100),
6     email VARCHAR(100),
7     phone_number VARCHAR(50),
8     registration_date VARCHAR(50),
9     status VARCHAR(20)
10 );
11
12 INSERT INTO source_data.customers (full_name, email, phone_number, registration_date, status) VALUES
13 ('ahmet Yılmaz', 'ahmet.y@example.com', '555-1234567', '2023/10/01', 'ACTIVE'),
14 (' MEHMET Kaya ', 'mehmet.k@ EXAmple.com', '+90 555 987 65 43', '01-11-2023', 'active'),
15 ('Ayşe Demir', NULL, '05551112233', '2023-12-05', 'INACTIVE'),
16 ('ahmet Yılmaz', 'ahmet.y@example.com', '555-1234567', '2023/10/01', 'ACTIVE'),
17 ('Fatma Sahin', 'fatmasahin@test.com', 'Bilinmiyor', '2024.01.15', 'pending'),
18 ('Ali Can', 'ali.can@example', '555 444 33 22', '15/02/2024', 'ACT');
```

```
ornek_veritabani=# select * from source_data.customers;
 id | full_name | email | phone_number | registration_date | status
-----+-----+-----+-----+-----+-----
  1 | ahmet Yılmaz | ahmet.y@example.com | 555-1234567 | 2023/10/01 | ACTIVE
  2 | MEHMET Kaya | mehmet.k@ EXAmple.com | +90 555 987 65 43 | 01-11-2023 | active
  3 | Ayşe Demir |  | 05551112233 | 2023-12-05 | INACTIVE
  4 | ahmet Yılmaz | ahmet.y@example.com | 555-1234567 | 2023/10/01 | ACTIVE
  5 | Fatma Sahin | fatmasahin@test.com | Bilinmiyor | 2024.01.15 | pending
  6 | Ali Can | ali.can@example | 555 444 33 22 | 15/02/2024 | ACT
(6 rows)
```

Bu işlem sonrasında şemadaki customer tablosu yukarıda görüldüğü hale gelmiştir. Ardından aşağıdaki transform.sql scripti çalıştırılmış ve veriler temizlenmiştir.

```
1 WITH DedupedCustomers AS (
2     SELECT *,
3     ROW_NUMBER() OVER(PARTITION BY full_name, email ORDER BY id) as rn
4     FROM source_data.customers
5 ),
6 CleanedText AS (
7     SELECT
8     id,
9     INITCAP(TRIM(REGEXP_REPLACE(full_name, '\s+', ' ', 'g'))) as clean_name,
10    COALESCE(LOWER(TRIM(REPLACE(email, ' ', ''))), 'unknown@domain.com') as clean_email,
11    REGEXP_REPLACE(phone_number, '\D', '', 'g') as raw_phone,
12    TRIM(registration_date) as raw_date,
13    CASE
14        WHEN LOWER(status) IN ('active', 'act') THEN 'ACTIVE'
15        WHEN LOWER(status) = 'inactive' THEN 'INACTIVE'
16        ELSE 'PENDING'
17    END as clean_status
18    FROM DedupedCustomers
19    WHERE rn = 1
20 );
```

```

20 FinalTransformation AS (
21     SELECT
22         id,
23         clean_name as full_name,
24         clean_email as email,
25         CASE
26             WHEN LENGTH(raw_phone) >= 10 THEN '+90' || RIGHT(raw_phone, 10)
27             ELSE 'INVALID_PHONE'
28         END as phone_number,
29         CASE
30             WHEN raw_date ~ '^\\d{4}[/.\\-]\\d{2}[/.\\-]\\d{2}$'
31             THEN TO_DATE(REPLACE(REPLACE(raw_date, '/', '-'), '.', '-'), 'YYYY-MM-DD')
32             WHEN raw_date ~ '^\\d{2}[/.\\-]\\d{2}[/.\\-]\\d{4}$'
33             THEN TO_DATE(REPLACE(REPLACE(raw_date, '/', '-'), '.', '-'), 'DD-MM-YYYY')
34             ELSE NULL
35         END as registration_date,
36
37         clean_status as status
38     FROM CleanedText
39 )
40 SELECT * FROM FinalTransformation;

```

Yukarıdaki bu scriptin işlevleri adımlar halinde aşağıda açıklanmıştır:

Adım 1: Bu adımda kaynak veri içerisinde tekrar eden satırlar işlemin dışında tutulur ve veri tabanında gereksiz yer tutması engellenir.

Adım 2: Bu adımda isimler içindeki gereksiz boşluklar silinir ve sadece baş harfleri büyük hale getirilir; e-postalar içindeki boşluklar silinir, tüm karakterler küçük hale getirilir ve boş bırakılan e-postalar yerine unknown@domain.com stringi yazılır; telefon numaralarındaki rakamlar dışındaki karakterler silinir; status değerleri ACTIVE, INACTIVE veya PENDING olacak şekilde standartlaştırılır.

Adım 3: Geçici isimler yerine olması gereken sütun adları yazılır, telefon numaraları uzunluk şartını karşılıyorsa ülke kodu eklenir, değilse INVALID_PHONE değeri yazılır; tarihler PostgreSQL DATE tipine dönüştürülür.

Adım 4: Temizlenmiş ve dönüştürülmüş veri ekrana basılır.

```

DatabaseScenarios-4 $ psql -U ismailbasaran -d ornek_veritabani < transform.sql
 id | full_name | email | phone_number | registration_date | status
-----+-----+-----+-----+-----+-----
  2 | Mehmet Kaya | mehmet.k@example.com | +905559876543 | 2023-11-01 | ACTIVE
  1 | Ahmet Yılmaz | ahmet.y@example.com | +905551234567 | 2023-10-01 | ACTIVE
  6 | Ali Can | ali.can@example | +905554443322 | 2024-02-15 | ACTIVE
  3 | Ayşe Demir | unknown@domain.com | +905551112233 | 2023-12-05 | INACTIVE
  5 | Fatma Sahin | fatmasahin@test.com | INVALID_PHONE | 2024-01-15 | PENDING
(5 rows)

```

Yukarıda görüleceği üzere veriler düzgün şekilde temizlenmiş ve dönüştürülmüştür.

2.3.2. Veri Yükleme

Yukarıda hazırlanan transform.sql scripti farklı bir dosyaya taşınmış ve başına aşağıdaki kısımlar eklenmiştir.

```
17 CREATE TABLE dest_db.customers (  
16     customer_key SERIAL PRIMARY KEY,  
15     source_id INT,  
14     full_name VARCHAR(100),  
13     email VARCHAR(100),  
12     phone_number VARCHAR(20),  
11     registration_date DATE,  
10     status VARCHAR(20),  
9     etl_load_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
8 );  
7  
6 INSERT INTO dest_db.customers (source_id, full_name, email, phone_number, registration_date, status)
```

Böylelikle temizlenen veri hedef veri tabanına yüklenmiştir.

Bu işlemler sonucunda hedef veri tabanımızda veriler aşağıdaki gibi görülmektedir.

```
DatabaseScenarios-4 $ psql -U ismailbasaran -d ornek_veritabani  
psql (18.3 (Homebrew))  
Type "help" for help.  
  
ornek_veritabani=# select * from dest_db.customers;  
 customer_key | source_id | full_name | email | phone_number | registration_date | status | etl_load_date  
-----  
1 | 2 | Mehmet Kaya | mehmet.k@example.com | +905559876543 | 2023-11-01 | ACTIVE | 2026-05-06 14:24:04.017713  
2 | 1 | Ahmet Yılmaz | ahmet.y@example.com | +905551234567 | 2023-10-01 | ACTIVE | 2026-05-06 14:24:04.017713  
3 | 6 | Ali Can | ali.can@example | +905554443322 | 2024-02-15 | ACTIVE | 2026-05-06 14:24:04.017713  
4 | 3 | Ayşe Demir | unknown@domain.com | +905551112233 | 2023-12-05 | INACTIVE | 2026-05-06 14:24:04.017713  
5 | 5 | Fatma Şahin | fatmasahin@test.com | INVALID_PHONE | 2024-01-15 | PENDING | 2026-05-06 14:24:04.017713  
(5 rows)
```

Veriler temizlenmiş ve dönüştürülmüş olarak başarılı şekilde hedef veri tabanına yüklenmiştir.

2.3.3. Raporların Oluşturulması

Veriler yüklendikten sonra aşağıdaki report.sql scripti hazırlanmıştır.

```
1 SELECT  
2     'Kaynak' as veri_asamasi,  
3     COUNT(*) as toplam_satir_sayisi,  
4     SUM(CASE WHEN email IS NULL OR email NOT LIKE '%@%' THEN 1 ELSE 0 END) as gecersiz_eposta_sayisi,  
5     SUM(CASE WHEN status NOT IN ('ACTIVE', 'INACTIVE', 'PENDING') THEN 1 ELSE 0 END) as standart_disi_statu_sayisi  
6 FROM source_data.customers  
7 UNION ALL  
8 SELECT  
9     'Temizlenmiş Veri' as veri_asamasi,  
10    COUNT(*) as toplam_satir_sayisi,  
11    SUM(CASE WHEN email IS NULL OR email NOT LIKE '%@%' THEN 1 ELSE 0 END) as gecersiz_eposta_sayisi,  
12    SUM(CASE WHEN status NOT IN ('ACTIVE', 'INACTIVE', 'PENDING') THEN 1 ELSE 0 END) as standart_disi_statu_sayisi  
13 FROM dest_db.customers;
```

Bu script kaynak veri ve hedef veri tabanındaki veri arasındaki kullanılamaz veri sayılarını kıyaslamaktadır. Script çalıştırıldığına alınan sonuç aşağıdadır.

```
DatabaseScenarios-4 $ psql -U ismailbasaran -d ornek_veritabani < report.sql  
 veri_asamasi | toplam_satir_sayisi | gecersiz_eposta_sayisi | standart_disi_statu_sayisi  
-----  
Kaynak | 6 | 1 | 3  
Temizlenmiş Veri | 5 | 0 | 0  
(2 rows)
```

Görüleceği üzere transform.sql scriptimiz veriyi temizleme ve dönüştürme işlemlerinde başarılı olmuştur.

2.4. Çıktılar

Bu projede gerçekleştirilen senaryo sonucunda büyük verilerde veri temizleme ve dönüştürme işlemlerinin nasıl yönetileceği, Window Functions, CTE ve Regex'in nasıl kullanılacağı öğrenilmiştir.

3. Veritabanı Yedekleme ve Otomasyon Çalışması

Raporun bu bölümünde veri tabanı yedekleme senaryosu işlenmektedir. Senaryo macOS ortamında PostgreSQL ve pgBackRest kullanılarak gerçekleştirilmiştir.

3.1. Projenin Amacı

Projenin amacı veri tabanı yedekleme işlemlerini otomatikleştirerek, veri tabanı yönetim süreçleri optimize edilir. Ayrıca yedeklerin düzenli olarak alındığını doğrulamak için yedek alınamaması durumunda yöneticilere bildirim gönderilir.

3.2. Projenin Kapsamı

Projenin kapsamı şu şekildedir:

- crontab ve bash scripting kullanarak yedekleme süreçlerini otomatikleştirmek.
- Yedekleme işlemi başarısız olduğunda yöneticileri bilgilendirecek servisleri kurmak.

3.3. Gerçekleştirim ve Yöntem

Raporun bu bölümünde senaryonun nasıl gerçekleştirildiği anlatılmaktadır.

3.3.1. Otomatik Yedekleme ve Uyarı Sistemi

Bu proje kapsamında vize için kullanılan database ve konfigrasyonlar kullanılmıştır. Bu kurulum süreci vize raporunda işlendiğinden tekrar işlenmeyecektir.

Proje doğrultusunda bir bash script dosyası oluşturulmuş ve içerişi aşağıdaki gibi doldurulmuştur.

```

1 PATH=/usr/local/bin:/opt/homebrew/bin:/usr/bin:/bin:$PATH
2
3 BACKUP_TYPE=$1
4 DATE=$(date +%Y-%m-%d_%H-%M-%S)
5 LOG_FILE="$HOME/Dev/DB_Scenarios/DatabaseScenarios-5/DB_Backups/backup_log.txt"
6
7 if [ "$BACKUP_TYPE" == "full" ]; then
8     echo "[${DATE}] Tam (Full) yedek baslatiliyor..." >> "$LOG_FILE"
9     pgbackrest --stanza=proje_db_stanza --config=/Users/ismailbasaran/Dev/DB_Scenarios/pgbackrest.conf --type=full backup
10 elif [ "$BACKUP_TYPE" == "diff" ]; then
11     echo "[${DATE}] Fark (Differential) yedeği baslatiliyor..." >> "$LOG_FILE"
12     pgbackrest --stanza=proje_db_stanza --config=/Users/ismailbasaran/Dev/DB_Scenarios/pgbackrest.conf --type=diff backup
13 else
14     echo "HATA: Geçerli bir yedek tipi belirtilmedi (full veya diff)."

```

Yukarıdaki scriptin işlevi şu şekildedir:

Adım 1: Yedekleme türünü, tarihi ve log dosyasını al.

Adım 2: Yedekleme türüne uygun pgBackRest komutunu çağır.

Adım 3: İşlem başarılı ise log dosyasına yaz. İşlem başarılı değilse log dosyasına yaz ve yöneticiye uyarı maili at.

```

0 2 * * 0 /Users/kullanici_adiniz/Desktop/advanced_backup.sh full
0 2 * * 1-6 /Users/kullanici_adiniz/Desktop/advanced_backup.sh diff
~

```

Sonrasında hazırlanan bu script macOS kendi görev zamanlama aracı olan crontab ile her Pazar gecesi saat 02.00'da tam yedek ve diğer günler gece 02.00'da fark yedeği alacak şekilde otomatize edilmiştir.

Yapılan bu işlemler sayesinde veri tabanının düzenli ve otomatik şekilde yedeklenmesi, yedekleme yapılamayan durumlarda ise yöneticilerin bilgilendirilmesi sağlanmıştır.

3.4. Çıktılar

Proje kapsamında yapılan araştırmalar ve geliştirmeler sonucunda PostgreSQL ile güvenli ve otomatize yedekleme sistemlerinin nasıl hazırlanacağı öğrenilmiştir.